Pashov Audit Group

Pump Security Review

February 9th 2026 - February 12th 2026

Contents

1. About Pashov Audit Group 3
2. Disclaimer 3
3. Risk Classification 3
4. About Pump 4
5. Executive Summary 4
6. Findings 5
Medium findings 6
[M-01] Pre-funding DoS on UserVolumeAccumulator PDA blocks cashback sell transactions 6
Low findings 8
[L-01] Closing `user_volume_accumulator` allows cashback without `claim_cashback` call 8
[L-02] UVA initialization occurs before writable check causing sell operation failure 8
[L-03] Creator vault rent exemption over transfer on `buy_exact_sol_in` 9
[L-04] Swap instructions ignore global cashback toggle 10
[L-05] User volume accumulator `user` not initialized on update 11

1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over $100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work here or reach out on Twitter @pashovkrum.

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive

4. About Pump

Pump on Solana is a platform for launching SPL coins that can be traded on a bonding curve without needing to provide initial liquidity. Once the coin reaches a particular market cap, liquidity is deposited from the bonding curve to Raydium, and the received LP tokens are burnt. Pump AMM is an AMM on the Solana blockchain.

5. Executive Summary

A time-boxed security review of the **pump-fun/pump-amm-2** and **pump-fun/pump-contracts-solana** repositories was done by Pashov Audit Group, during which **0xAlix2**, **Rayaa**, **newspace**, **ZeroTrust01** engaged to review **Pump**. A total of **6** issues were uncovered.

Protocol Summary	
Project Name	Pump
Protocol Type	AMM and Bonding Curve tokensale
Timeline	February 9th 2026 - February 12th 2026

Review commit hashes:

- 9b9fc98d469243bd0e96dfefbf826137732c435c (pump-fun/pump-amm-2)
- 164bf9043dc94f736474c73126da5b654c2019b1 (pump-fun/pump-contracts-solana)

Fixes review commit hash:

- b2ff0bbb9912ec2c13f178e636258103cd23b979 (pump-fun/pump-amm-2)

Scope

`mod.rs` `toggle_cashback_enabled.rs` `buy.rs` `sell.rs` `claim.rs` `init_user_volume_accumulator.rs` `create_pool.rs` `extend_account.rs` `global_config.rs` `pool.rs` `volume_accumulator.rs` `token.rs` `common.rs` `lib.rs` `claim_token_incentives.rs` `create_v2.rs` `migrate.rs`

6. Findings

Findings count

Severity	Amount
Medium	1
Low	5
Total findings	**6**

Summary of findings

ID	Title	Severity	Status
[M-01]	Pre-funding DoS on UserVolumeAccumulator PDA blocks cashback sell transactions	Medium	**Resolved**
[L-01]	Closing `user_volume_accumulator` allows cashback without `claim_cashback` call	Low	Acknowledged
[L-02]	UVA initialization occurs before writable check causing sell operation failure	Low	**Resolved**
[L-03]	Creator vault rent exemption over transfer on `buy_exact_sol_in`	Low	Acknowledged
[L-04]	Swap instructions ignore global cashback toggle	Low	Acknowledged
[L-05]	User volume accumulator `user` not initialized on update	Low	**Resolved**

Medium findings

[M-01] Pre-funding DoS on UserVolumeAccumulator PDA blocks cashback sell transactions

Severity

Impact: High

Likelihood: Low

Description

Both `pump-contracts-solana` and `pump-amm-2` use `system_program::create_account` to initialize `UserVolumeAccumulator` PDA accounts during sell transactions for cashback-enabled coins:

- `pump-contracts-solana/programs/pump/src/sell.rs:48` in `try_process_user_volume_accumulator()`
- `pump-amm-2/programs/pump-amm/src/instructions/swap/sell.rs:318` in `try_process_user_volume_accumulator_sell_cashback()`

The Solana runtime's `create_account` instruction checks whether the destination account already has a non-zero lamport balance. If `lamports > 0`, it rejects the call with `AccountAlreadyInUse`, regardless of whether the account has data or an assigned owner.

An attacker can exploit this by:

1. Precomputing the victim's `UserVolumeAccumulator` PDA address (seeds: `USER_VOLUME_ACCUMULATOR_SEED + user_pubkey`, fully predictable).
2. Transferring a small amount of lamports (e.g., minimum rent-exempt balance for a 0-data system account) to the PDA address.

After pre-funding, the PDA has `lamports > 0`, `data_len == 0`, `owner == System Program`. When the victim attempts to sell a cashback-enabled coin:

- `data_is_empty()` returns `true` (since `data_len == 0`), so the code enters the initialization branch.
- `create_account` fails with `AccountAlreadyInUse` because `lamports > 0`.
- The error propagates via the `?` operator, reverting the entire transaction.

Although both call sites are wrapped in `try_process_user_volume_accumulator` functions that return `Result<bool>` with a fallback path (redirecting `creator_fee` to `creator_vault` / `coin_creator_vault_ata`), the `?` on `create_account` causes the error to propagate upward before the fallback can execute. This means the sell transaction reverts entirely, not just the cashback portion.

This DoS is **persistent** — once an address is pre-funded, all subsequent sell attempts for cashback coins by the targeted user will fail until the issue is resolved. The attacker can target any user at minimal cost.

Recommendations

Replace `system_program::create_account` with the safer `transfer + allocate + assign` pattern that tolerates pre-funded accounts:

```
let current_lamports = user_volume_accumulator_info.lamports();
if current_lamports == 0 {
    // Safe to use create_account
    system_program::create_account(/* ... */)?;
} else {
    // Pre-funded: top up to rent-exempt, then allocate + assign
    let required = rent.minimum_balance(space).saturating_sub(current_lamports);
    if required > 0 {
        system_program::transfer(/* payer -> PDA, required */)?;
    }
    system_program::allocate(/* PDA, space, signer_seeds */)?;
    system_program::assign(/* PDA, &crate::ID, signer_seeds */)?;
}
```

Alternatively, catch the `create_account` error gracefully and fall through to the existing fallback path instead of propagating it with `?`.

Low findings

[L-01] Closing `user_volume_accumulator` allows cashback without `claim_cashback` call

Description

In the pump program, cashback is lamports on the `user_volume_accumulator` PDA. `close_user_volume_accumulator` uses `close = user`, so all PDA lamports (including cashback) are sent to the user when the account is closed.

```
    mut,
    seeds = [USER_VOLUME_ACCUMULATOR_SEED, user.key().as_ref()],
    bump,
    close = user,
)]
pub user_volume_accumulator: Account<'info, UserVolumeAccumulator>,
```

That lets users receive cashback without calling `claim_cashback_ix`, so the intended claim is bypassed.

Recommendations:

Consider checking if there is any outstanding cashback in the accumulator before allowing the user to close it.

[L-02] UVA initialization occurs before writable check causing sell operation failure

Description

The function `try_process_user_volume_accumulator_sell_cashback` is designed to optionally process cashback during a sell operation. If the `UserVolumeAccumulator` (UVA) PDA or its ATA is invalid or improperly provided, the function should return `Ok(false)` so the caller can fall back to transferring the fee to `coin_creator_vault_ata`.

However, when the UVA PDA does not exist `(data_is_empty())`, the function calls `system_program::create_account` before checking whether the account is **writable**.

If the correct UVA PDA is passed but not marked as writable in the transaction, the `create_account` CPI fails with `AccountNotWritable`, causing the entire sell instruction to revert. As a result, the fallback logic in `sell()` is never executed.

This breaks the intended optional behavior of cashback handling and causes an unexpected transaction failure for first-time sellers.

Impact

For cashback coins, a first-time seller who provides the correct UVA PDA but forgets to mark it writable will experience a full transaction revert instead of a successful sell with a fallback fee transfer.

No funds are lost, but the sell operation becomes unexpectedly unexecutable in this scenario.

Affected Code

- https://github.com/PashovAuditGroup/scope_pump-fun_pump-amm-2_08-02-2026/blob/f74098d909473c3826e4f6bde65965f6801792a2/programs/pump-amm/src/instructions/swap/sell.rs#L299-L344

Recommendations

Move the writable check before the account initialization logic:

```
if !user_volume_accumulator_account_info.is_writable {
    return Ok(false);
}

if user_volume_accumulator_account_info.data_is_empty() {
    system_program::create_account(...)?;
}
```

[L-03] Creator vault rent exemption over transfer on `buy_exact_sol_in`

Description

When a user buys using `buy_exact_sol_in` on a non–cashback bonding curve with `creator_fee > 0`, the program now sends creator fees to the creator vault in two steps:

1. `make_creator_vault_rent_exempt(system_program, user, creator_vault)?` tops up the vault to rent exemption.
2. `transfer_sol_from_user(system_program, user, creator_vault, creator_fee)?` sends the creator fee.

`make_creator_vault_rent_exempt` does the following when the vault is below rent exemption:

- It computes `minimum_rent_exemption_balance = Rent::get()?.minimum_balance(CREATOR_VAULT_DATA_LEN)`.
- It then transfers that **full** amount to the vault: `transfer_sol_from_user(..., minimum_rent_exemption_balance)`.
- It does **not** subtract the vault's current lamports. So if the vault already has some SOL, the user still pays the full rent exemption amount.

This differs from the previous behavior, where the user was only supposed to top up the rent instead of paying the whole rent.

`make_creator_vault_rent_exempt` does the following when the vault is below rent exemption:

- Vault already has 300 lamports; rent exemption is 1,000; `creator_fee` is 500.
- Current behavior: `make_creator_vault_rent_exempt` transfers 1,000; then `creator_fee` transfer adds 500. Total user cost is 1,500; vault ends at 300 + 1,000 + 500 = 1,800 lamports (300 extra).
- Old behavior: only 700 lamports are needed for rent (1,000 − 300). User should pay 700 + 500 = 1,200 in a single transfer; vault ends at 1,500 (exactly rent + fee).

This forces the buyer to overpay SOL and could lead to compatibility issues.

Recommendations

Consider restoring the previous behavior to ensure that buyers do not overpay.

[L-04] Swap instructions ignore global cashback toggle

Description

The `buy` and `sell` swap instructions execute cashback logic based only on the pool-level flag (is_cashback_coin). They do not verify the global configuration flag `global.is_cashback_enabled` at runtime.

At pool creation, `global.is_cashback_enabled` is checked before marking a pool as a cashback pool. However, this acts only as a creation-time gate. If the admin later disables cashback using `toggle_cashback_enabled(false)`, the global flag becomes `false`, but existing pools still retain `is_cashback_coin = true`.

In sell.rs, when `creator_fee > 0`, the code checks only `bonding_curve.is_cashback_coin` before executing `try_process_user_volume_accumulator`. There is no validation of `global.is_cashback_enabled`. The same issue exists in the AMM repository swap logic.

As a result, disabling cashback globally does not stop cashback execution for already created pools. The global toggle does not function as a true runtime kill switch.

The logic has been reviewed and no additional runtime guard prevents cashback execution when the global flag is false. The issue is confirmed.

Affected Code

- https://github.com/PashovAuditGroup/scope_pump-fun_pump-amm-2_08-02-2026/blob/f74098d909473c3826e4f6bde65965f6801792a2/programs/pump-amm/src/instructions/swap/sell.rs#L185
- https://github.com/PashovAuditGroup/scope_pump-fun_pump-amm-2_08-02-2026/blob/f74098d909473c3826e4f6bde65965f6801792a2/programs/pump-amm/src/instructions/swap/buy.rs#L377
- https://github.com/PashovAuditGroup/scope_pump-fun_pump-contracts-solana_08-02-2026/blob/b2098c2405eff277bee9871bb174bdc9bfeb0d68/programs/pump/src/lib.rs#L617-L829

Impact

Global cashback disable does not affect existing pools. Cashback PDAs can still be funded and state can still be mutated, weakening administrative control and emergency response capabilities.

Recommendations

Enforce cashback execution in both Bonding Curve and AMM swap logic only when both the pool-level `is_cashback_coin` flag and the global `is_cashback_enabled` flag are `true`; otherwise, route creator fees to the standard creator vault.

[L-05] User volume accumulator `user` not initialized on update

Description

In `pump-contracts-solana` sell path, when an existing `UserVolumeAccumulator` account is used, the `user` field is not (re)initialized or verified. This can leave `user` unset, which can break off-chain attribution or any on-chain logic that relies on the `user` field being populated.

Recommendations

When processing an existing accumulator, set the user if it is still the default value, and optionally validate that it matches the signer to prevent stale or incorrect attribution.

```
++  if user_volume_accumulator.user == Pubkey::default() {
      user_volume_accumulator.user = user.key();
++  }
```